

TEMA 2 – Objetos en JavaScript

Objetivo del tema

En este tema pretende ser un resumen de lo que es un Objeto en JavaScript.

2.1 Constructores

Un **objeto** en JavaScript, desde ES6, utiliza la sintaxis de las clases típicas de Java para ser definidos. Nótese que por debajo siguen utilizando **prototipos**, sólo se ha ‘mejorado’ la sintaxis para hacerlo todo más limpio.

```
class Vehiculo {  
    // El único constructor de Vehículo  
    constructor(marca, modelo, kilometraje = 0) {  
        this.marca = marca;           // Propiedad pública  
        this.modelo = modelo;         // Propiedad pública  
        this.kilometraje = kilometraje;  
    }  
}  
  
// Instanciación del objeto  
const miAuto = new Vehiculo("Toyota", "Corolla", 15000);
```

JavaScript **NO** permite que haya más de un constructor por objeto. Nótese que se hace innecesario declarar los campos **públicos** fuera del constructor.

Por defecto, es posible acceder a las propiedades y métodos de la clase mediante “.” o “[]”:

- “.” Cuando conoces el nombre de la propiedad en tiempo de ejecución.
- “[]” Cuando la clave es dinámica o contiene caracteres especiales.

```
class Calculadora {  
    sumar(a, b) { return a + b; }  
    restar(a, b) { return a - b; }  
}  
  
const calc = new Calculadora();  
  
// Acceso estándar por punto  
console.log(calc.sumar(5, 3)); // 8  
  
// Acceso dinámico por corchetes  
const operacion = "restar";  
console.log(calc[operacion](10, 4)); // 6
```

2.2 Publico & Privado

Históricamente se usaba la convención de guion bajo (`_propiedad`) para indicar privacidad, pero **no impedía el acceso externo**. En JavaScript moderno (ES2022+), los miembros privados se definen usando el prefijo `#`. Además, un campo **privado** siempre se debe declarar fuera del constructor.

Tomemos, por ejemplo:

```
class CuentaBancaria {
  // Es obligatorio declarar los campos privados fuera del constructor
  #saldo;
  #pin;

  constructor(titular, saldoInicial, pin) {
    this.titular = titular;    // Público
    this.#saldo = saldoInicial; // Privado
    this.#pin = pin;          // Privado
  }

  // Método público que accede a lo privado
  obtenerSaldo(pinIngresado) {
    if (this.#validarPin(pinIngresado)) {
      return this.#saldo;
    }
    throw new Error("PIN incorrecto.");
  }

  // Método privado
  #validarPin(pin) {
    return this.#pin === pin;
  }
}
```

Luego, podemos hacer:

```
const miCuenta = new CuentaBancaria("Borja", 1000, 1234);

console.log(miCuenta.titular); // "Borja"
console.log(miCuenta.obtenerSaldo(1234)); // 1000
```

Sin embargo, esto genera errores:

```
console.log(miCuenta.#saldo);
// SyntaxError: Private field '#saldo' must be declared
// in an enclosing class
```

2.3 Getters & Setters

En JavaScript, **NO** tiene sentido definir los getters y setters de todas las propiedades. Sólo deberías añadir uno cuando **aportan lógica** explícita.

Es decir, cuando a parte de hacer un get o un set además hacen otra cosa. Lo cual, por cierto, contradice el Principio de Responsabilidad Única de los principios SOLID.

... de todas formas, y puestos a imitar a Java, podemos hacer algo así:

```
class Usuario {
  #nombre;

  constructor(nombre) {
    this.#nombre = nombre;
  }

  // Getter: Se invoca SIN paréntesis como si fuera una propiedad
  get nombre() {
    return this.#nombre.toUpperCase();
  }

  // Setter: Recibe exactamente UN parámetro y permite validar
  set nombre(nuevoNombre) {
    if (typeof nuevoNombre !== "string" || nuevoNombre.trim() === "") {
      throw new Error("El nombre no es válido.");
    }
    this.#nombre = nuevoNombre;
  }
}
```

Y entonces, hacer:

```
const user = new Usuario("lucia");

// Uso del Getter (sin paréntesis)
console.log(user.nombre); // "LUCIA"

// Uso del Setter (asignación directa)
user.nombre = "Marta";
console.log(user.nombre); // "MARTA"
```

2.4 Herencia

En JavaScript, existe un mecanismo de Herencia similar a Java para extender la funcionalidad de una clase padre. Aunque la mecánica existe, dada la naturaleza de JavaScript (código pequeño, compacto, funcional), puede que no tenga un uso tan habitual como Java. Como ejemplo:

```
// Clase Padre
class Empleado {
  constructor(nombre, sueldoBase) {
    this.nombre = nombre;
    this.sueldoBase = sueldoBase;
  }

  calcularNomina() {
    return this.sueldoBase;
  }
}

// Clase Hija
class Desarrollador extends Empleado {
  constructor(nombre, sueldoBase, lenguaje) {
    // Obligatorio llamar a super() ANTES de usar 'this'
    super(nombre, sueldoBase);
    this.lenguaje = lenguaje;
  }

  programar() {
    console.log(`${this.nombre} está escribiendo código en ${this.lenguaje}.`);
  }
}
```

Así podemos hacer:

```
const dev = new Desarrollador("Aitor", 2500, "JavaScript");
console.log(dev.calcularNomina()); // 2500
dev.programar();
```

JavaScript también permite **polimorfismo**, que distintas clases hijas respondan al mismo método con comportamientos específicos. Para sobrescribir un método, simplemente se vuelve a definir en la subclase.

2.5 Herencia

Podemos definir miembros o propiedades **estáticas**. Estos pertenecen a la **clase en sí** y no a las instancias de la clase. No pueden acceder a las propiedades de instancia mediante `this`, sino mediante `“.”`.

```
class ConvertidorMatematico {
  static PI = 3.14159;

  static celsiusAFahrenheit(celsius) {
    return (celsius * 9/5) + 32;
  }
}
```

Y entonces, hacemos:

```
console.log(ConvertidorMatematico.PI);
console.log(ConvertidorMatematico.celsiusAFahrenheit(25));

const conv = new ConvertidorMatematico();
conv.celsiusAFahrenheit(25); // ERROR
```